

This architecture is designed for high availability, consistency, and regulatory compliance. Given the requirements, we will adopt a **Microservices Architecture** orchestrated by Kubernetes, leveraging an Event-Driven pattern for asynchronous processing.

1. Architecture Overview

- **API Gateway:** Handles SSL termination, Rate Limiting, and JWT validation.
- **Communication:** REST/gRPC for synchronous tasks; Apache Kafka for asynchronous event-driven workflows (e.g., fraud detection triggers, transaction notifications).
- **Infrastructure:** Multi-AZ deployment in a Private Cloud VPC.

2. Services

- **Identity Service:** User registration, MFA orchestration (TOTP/SMS), RBAC.
- **Account Service:** Ledger operations, balance checks, account lifecycle management.
- **Transaction Service:** Orchestrates funds transfer, handles idempotency, and connects to the Fraud Detection engine.
- **Loan Service:** Workflow management for applications, credit score integration, and approval state machine.
- **Analytics Service:** Read-only access to transaction history for AI-driven spend analysis.
- **Audit Service:** High-performance, write-only service consuming audit events from the message bus.

3. Modules (Service Level)

- **Persistence Layer:** Repository pattern with ORM or SQL Query Builders.
- **Business Logic Layer:** Service classes implementing domain rules.
- **Integration Layer:** Adapters for external APIs (Credit Bureau, Notification providers).
- **Security Wrapper:** Decorators for input validation and audit logging.

4. Authentication & Authorization

- **Authentication:** OAuth2 + OIDC with JWTs. MFA enforced at the API Gateway via "step-up" authentication (returning `403` with a challenge requirement).

-
- **Authorization:** Attribute-Based Access Control (ABAC). Roles (Admin, Staff, Customer) define permissions, but constraints (e.g., "Owner of Account") are checked via logic.

5. Business Logic

- **Idempotency:** All transactions require an `idempotency-key` in headers to prevent duplicate charges during network retries.
- **Saga Pattern:** Use for distributed transactions (e.g., deducting from A and adding to B) to ensure eventual consistency.

6. Background Jobs

- **Scheduler:** Distributed Task Queue (e.g., BullMQ or Celery) for:

* Refreshing AI budget insights (daily).

* Processing pending loan maturity checks.

* Cleanup of expired MFA tokens.

7. File Storage

- **Object Storage:** S3-compatible storage for encrypted document uploads (KYC documents, loan agreements).
- **Data Integrity:** File hashes stored in the database to detect tampering.

8. Error Handling & Logging

- **Error Handling:** Global exception handler returning RFC 7807 compliant error objects.
- **Logging:** Centralized Logging (ELK Stack/Grafana Loki). Logs must be scrubbed of PII/PCI data.
- **Tracing:** Distributed tracing (Jaeger/OpenTelemetry) to track a single transaction across services.

9. Recommended Folder Structure

```
/src
/api          # Controllers/Resolvers
/application  # Use cases/Services
/domain       # Entities, Interfaces, Value Objects
/infrastructure # Database, External APIs, Message Bus
/shared       # Common utilities, Middlewares, Types
/tests        # Unit, Integration, E2E
/k8s          # Helm charts/Deployment manifests
```

10. Recommended Technology Stack

- **Language:** Go (High performance/Concurrency) or Java (Spring Boot - robust banking enterprise support).
- **Database:** PostgreSQL (with Row-Level Security for multi-tenancy/isolation).
- **Cache:** Redis (Rate limiting and session storage).
- **Messaging:** Apache Kafka (High throughput/Immutable event logs).
- **Monitoring:** Prometheus/Grafana (Metrics), Sentry (Error tracking).
- **Security:** HashiCorp Vault (Secret management).

11. Security Architecture (Specific Constraints)

- **PCI-DSS:** Segregate the Transaction Service into a isolated "CDE" (Cardholder Data Environment).
- **Auditability:** Every state change in the `Transactions` and `Loans` tables must be captured in the `Audit\_Logs` using database triggers or Change Data Capture (CDC) via Debezium.
- **Fraud Detection:** An asynchronous consumer triggers the `fraud\_engine` as soon as a `TXN\_CREATED` event hits Kafka; if a high score is returned, the transaction status is updated to `FLAGGED` before funds are moved.